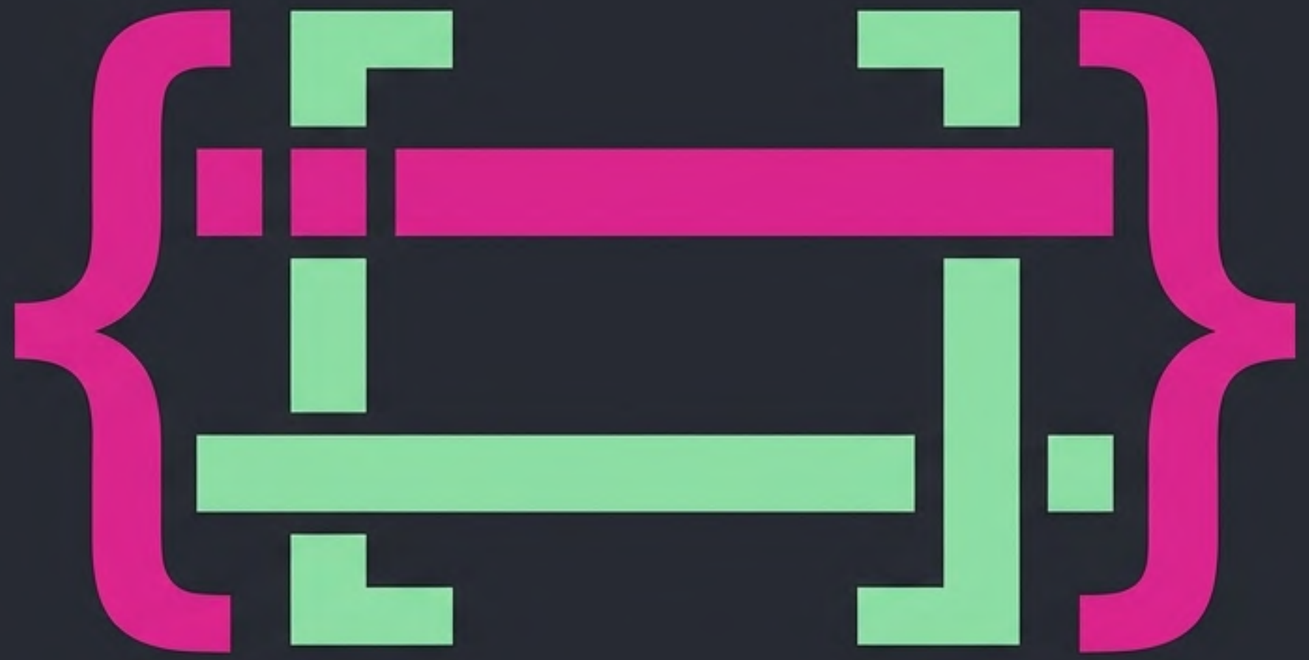
index.js



Funciones y Arrays:

Dominando la Lógica y los Datos en JavaScript

De la construcción de acciones aisladas a la manipulación masiva de colecciones.

NotebookLM

Funciones: Los Bloques de Construcción

Las funciones son las acciones (los verbos) de tu programa. Permiten encapsular lógica para ejecutarla múltiples veces sin duplicar código. Si necesitas cambiar la acción, solo modificas un lugar.

script.js

```
1  // 1. Declaración de la función
2  function showMessage() {
3    alert(''¡Hola a todos!');
4  }
5
6  // 2. Ejecución (Llamada)
7  showMessage();
8  showMessage(); // Sin duplicar código
```

Reutilización

NotebookLM

Anatomía de una Función

Nombre (El Verbo)

Parámetros y Valores por Defecto

Palabra clave

script.js

```
function sum(a, b = 0) {
  return a + b;
}

let result = sum(5, 10);
```

Valor de Retorno

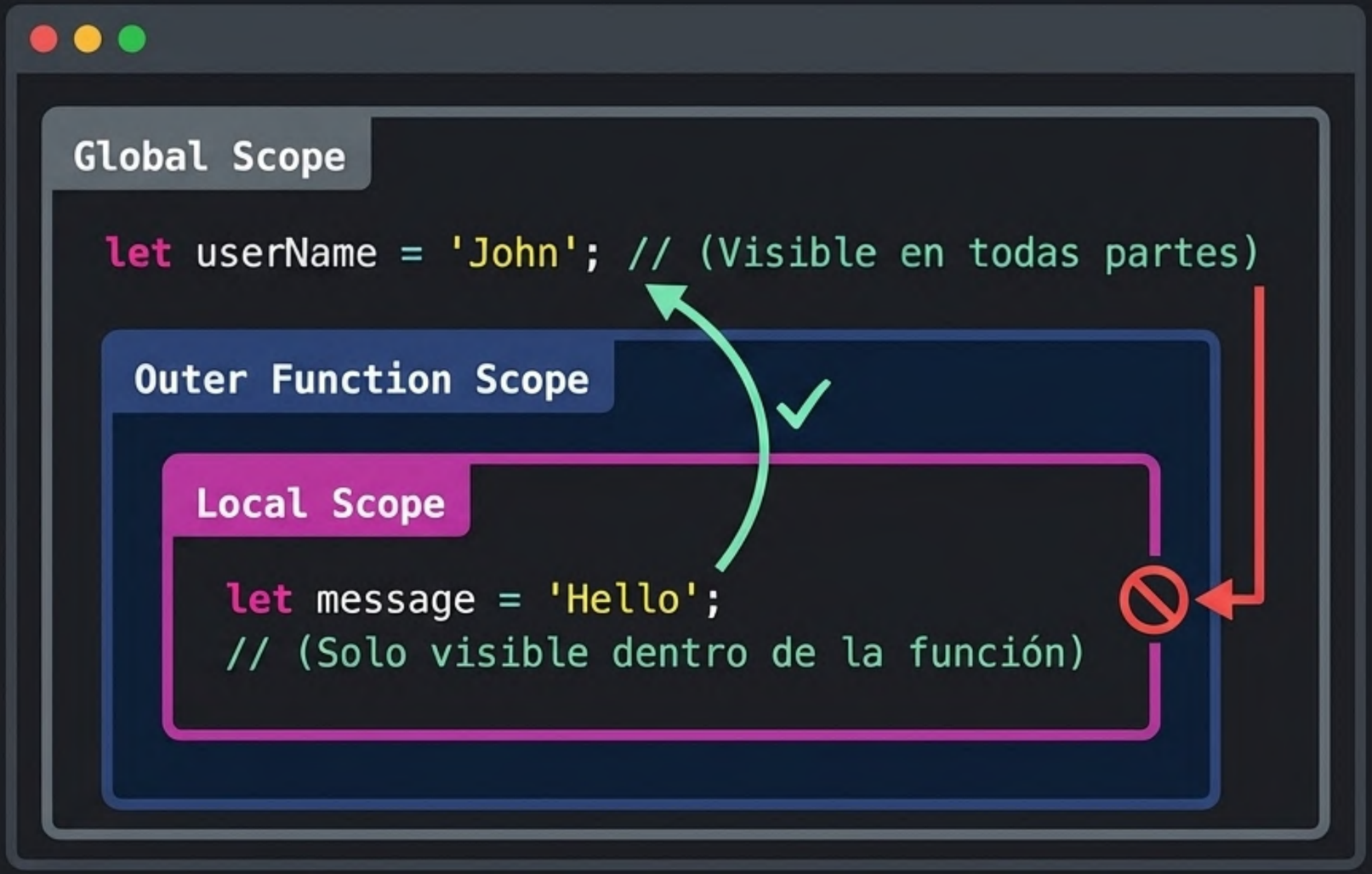
Argumentos

Un *return* vacío devuelve undefined. Los parámetros por defecto (= 0) protegen tu función cuando faltan argumentos.

NotebookLM

El Ecosistema de las Variables (Scope)

Una función puede acceder a las variables externas, pero el exterior nunca puede ver las variables locales.



Shadowing (Sombreado)
Si una variable local tiene el mismo nombre que una externa, la función ignora la externa y usa su propia copia local.

NotebookLM

Código Limpio: Las Funciones son Comentarios

El nombre de una función debe ser un verbo claro que describa exactamente una acción. Una función, un propósito.

Prefijo (Monospace)	Descripción y Ejemplo
`show...`	Muestra algo en la interfaz (ej. `showMessage`)
`get...`	Devuelve un valor (ej. `getAge`)
`calc...`	Realiza una operación matemática (ej. `calcSum`)
`create...`	Instancia un nuevo objeto o elemento (ej. `createForm`)
`check...`	Evalúa una condición y devuelve un booleano (ej. `checkPermission`)

❌

```
function getAgeAndShow() { ... }
```

✅

```
function getAge() { ... }
function showAge() { ... }
```

Regla de Oro: Divídelas para que cada una haga solo una cosa.

NotebookLM

Transición: De la Lógica a los Datos

Mientras las funciones ordenan nuestras acciones, los Arrays ordenan nuestros datos. Son colecciones indexadas diseñadas para almacenar listas secuenciales.

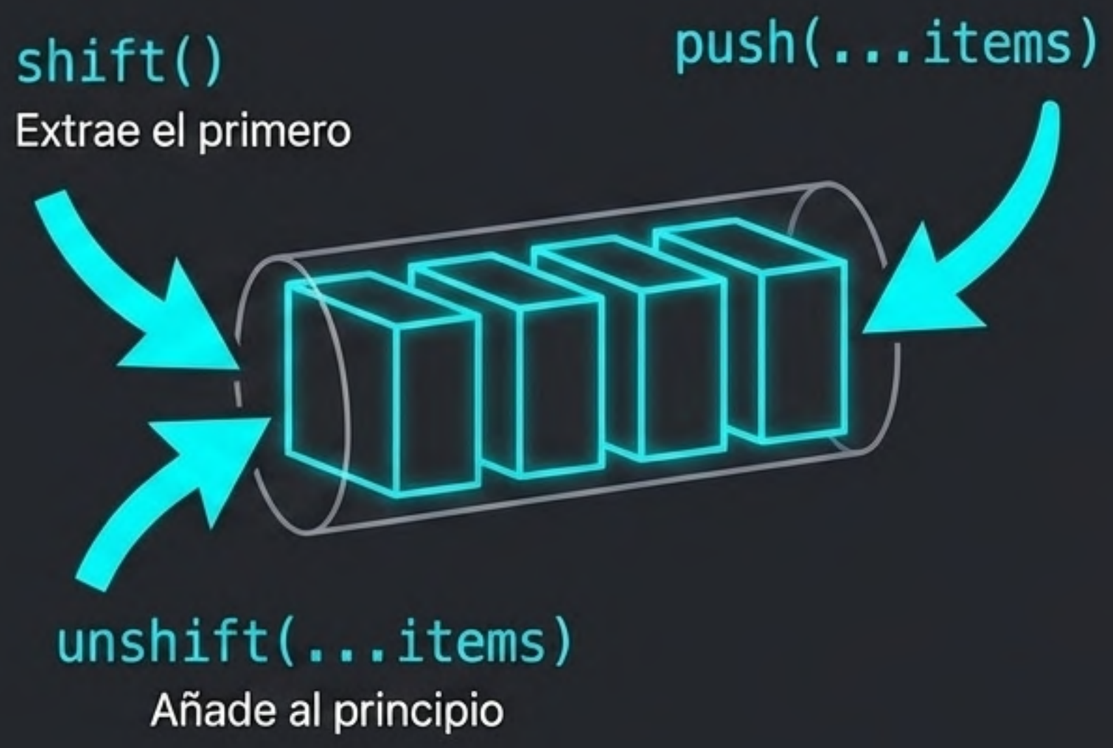
```
let fruits = ['Apple', 'Orange', 'Plum'];

// El método tradicional (verboso)
let last = fruits[fruits.length - 1];

// El método moderno (elegante)
let modernLast = fruits.at(-1); // 'Plum'
```

NotebookLM

Pilas y Colas: Manipulando los Extremos

Stack (LIFO - Último en entrar, primero en salir)

Queue (FIFO - Primero en entrar, primero en salir)



NotebookLM

El Costo del Rendimiento: Re-indexación

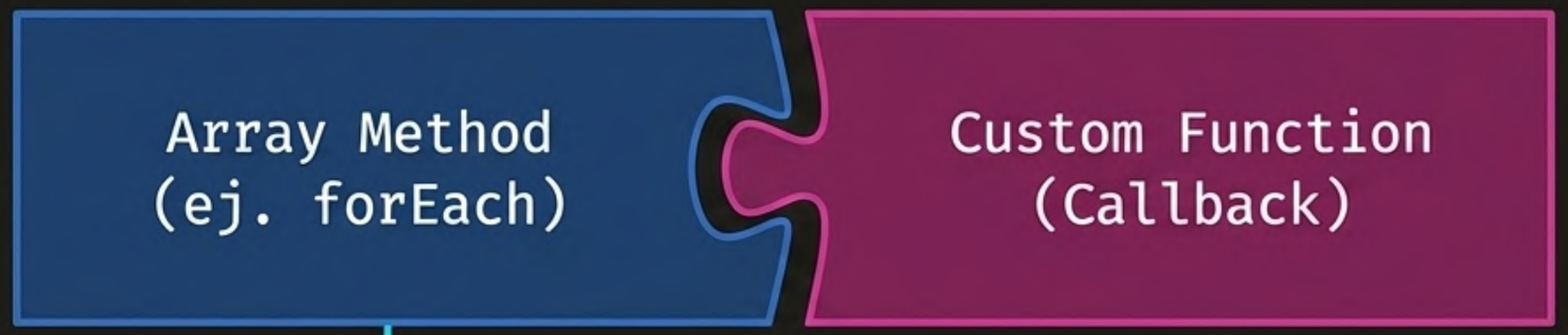
Trabajar con el final del array es instantáneo.
Trabajar con el principio es costoso.

Al eliminar o añadir en el índice 0, el motor debe re-numerar físicamente todos los demás elementos y actualizar el “length”.

NotebookLM

El Puente: Callbacks

(Donde la Lógica conoce a los Datos)




```
['Bilbo', 'Gandalf', 'Nazgul'].forEach((item, index) => {
  alert(`${item} está en el índice ${index}`);
});
```

El Método: Controla el bucle (cuándo y cómo iterar).

El Callback: Controla la acción (qué hacer con cada dato).

NotebookLM

Matriz de Modificación (Mutabilidad)

La regla de oro para evitar bugs: saber qué métodos alteran los datos originales y cuáles crean copias seguras.

⚠️ Muta el Array Original	✅ Devuelve un Nuevo Array
`splice(start, count, ...items)` 🪚 La navaja suiza. Inserta, elimina o reemplaza.	`slice(start, end)` Copia segura de un sub-array.
`sort(fn)` Ordena in-place.	`concat(...items)` Une arrays.
`reverse()` Invierte in-place.	`map(fn)` Transforma elementos en un array nuevo.
`push, pop, shift, unshift` Añaden o eliminan en los extremos.	`filter(fn)` Filtra elementos en un array nuevo.

NotebookLM

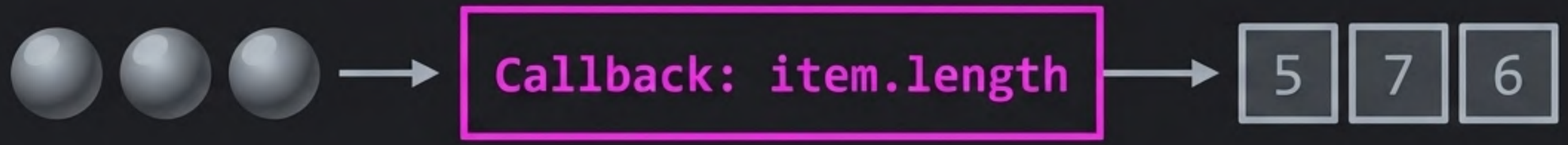
Matriz de Búsqueda: Eligiendo la Herramienta Correcta

Lo que necesitas (Intent)	**El Método** (Tool)	**Detalle Clave** (Nuance)
¿Necesitas el índice (posición)?	`indexOf()` / `findIndex(fn)`	Devuelve -1 si no existe.
¿Necesitas saber si existe (Sí/No)?	`includes()`	Maneja correctamente NaN (indexOf no lo hace).
¿Necesitas un elemento específico?	`find(fn)`	Se detiene y devuelve el primer elemento que cumpla la condición.
¿Necesitas múltiples elementos?	`filter(fn)`	Devuelve un nuevo array con todas las coincidencias.

NotebookLM

Transformación de Datos: map y sort

map(fn) – Transformación 1-a-1



Devuelve un array nuevo con los resultados exactos.

El Peligro de sort() – Ordenamiento Léxico

¡Por defecto, JavaScript ordena todo como Strings! Lexicográficamente, '15' es menor que '2'.

[1, 2, 15] → [1, 15, 2]

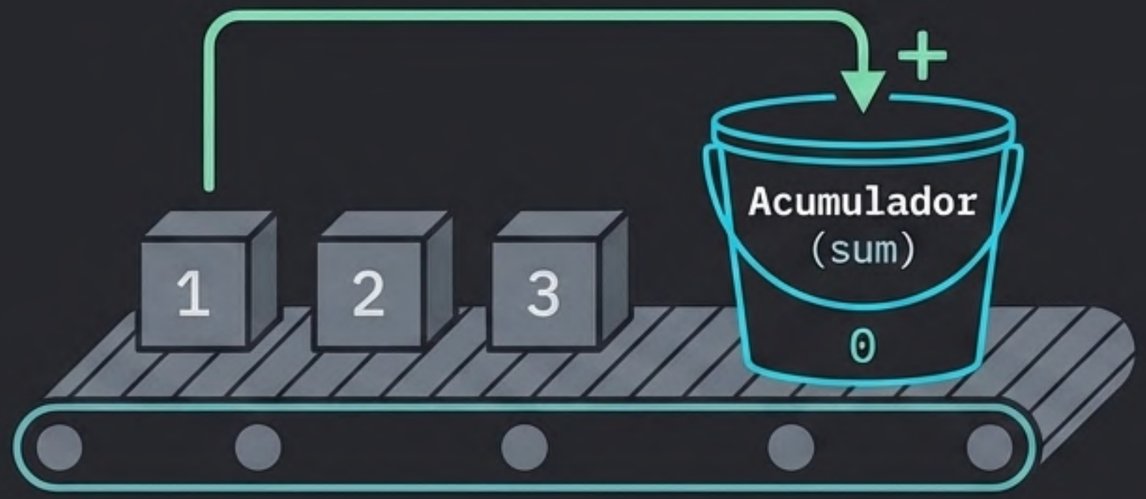
```
let arr = [1, 2, 15];
arr.sort((a, b) => a - b); // [1, 2, 15] correcto
```

NotebookLM

El Motor de Acumulación (reduce)

Calcula un único valor basado en todo el array. Transfiere su resultado de una iteración a la siguiente a través del Acumulador.

```
let arr = [1, 2, 3];
let result = arr.reduce((sum, current) =>
  sum + current, 0);
```



Paso	sum (Acumulador)	current	Resultado
Llamada 1	0	1	1
Llamada 2	1	2	3
Llamada 3	3	3	6

Warning: Siempre especifica un valor inicial (ej. 0) para evitar errores si el array está vacío.

NotebookLM

Cheat Sheet Visual: El Ecosistema de los Arrays

Añadir / Eliminar

Extremos: push, pop, shift, unshift
Cualquier posición: splice
Copiar sub-sección: slice

Buscar

Posición: indexOf, findIndex
Booleano: includes, some, every
Elemento(s): find, filter

Transformar

1 a 1: map
Único valor: reduce
Modificar orden: sort, reverse
Strings: split, join

Utilidades

Iteración simple: forEach
Validación de tipo:
Array.isArray(value)

NotebookLM

Lógica Fluida, Datos Estructurados

```
const mastery = (logic, data) => {
  // Las funciones encapsulan tu intención
  const builtLogic = buildFunctions(logic);

  // Los arrays y sus métodos ejecutan esa intención
  return data.map(builtLogic);
};

// El mejor código no es el más corto.
// Es el más fácil de leer mañana.
```

IDE Terminal

```
> _ Aprendizaje completado. Listo para escribir código limpio.
Process finished with exit code 0
```

NotebookLM